ÖĞRENCİLERİN AKADEMİK BAŞARILARINI ETKİLEYEN FAKTÖRLERİN VERİ MADENCİLİĞİ VE MAKİNE ÖĞRENMESİ YÖNTEMLERİYLE İNCELENMESİ

Sude Ateş(2322120104)

ÖZET

Bu çalışmada, öğrencilerin akademik başarılarını etkileyen faktörlerin incelenmesi ve matematik başarı düzeylerinin tahmin edilmesi amaçlanmıştır. Çalışmada Kaggle platformunda yer alan *Students Performance* veri seti kullanılmıştır. Veri seti; öğrencilerin cinsiyet, ebeveyn eğitim durumu, öğle yemeği tipi, test hazırlık durumu gibi demografik ve sosyo-ekonomik özelliklerini içermektedir. Hedef değişken olarak matematik notları ele alınmış ve bu notlar düşük ve yüksek başarı olmak üzere iki sınıfa ayrılmıştır.Veri ön işleme aşamasında kategorik değişkenler sayısal değerlere dönüştürülmüş ve veri eğitim ile test kümelerine ayrılmıştır. Modelleme sürecinde K-En Yakın Komşu (KNN), Naive Bayes, Decision Tree, Support Vector Machine (SVM), Logistic Regression ve KMeans algoritmaları kullanılarak performans karşılaştırması yapılmıştır. Modellerin başarımı doğruluk (accuracy) metriği ve hata matrisleri (confusion matrix) ile değerlendirilmiştir. Ayrıca veri setindeki değişkenler arasındaki ilişkiler korelasyon analizi ile incelenmiştir.Elde edilen sonuçlara göre, kullanılan modeller arasında en yüksek doğruluk oranını veren model en iyi performans gösteren model olarak seçilmiştir. Çalışma, öğrencilerin matematik başarısının tahmin edilebileceğini ve farklı makine öğrenmesi yöntemlerinin performanslarının karşılaştırmalı olarak değerlendirilebileceğini göstermektedir.

Anahtar Kelimeler: Öğrenci Performansı,Veri Madenciliği,Makine Öğrenmesi,Sınıflandırma,Lojistik Regresyon,Doğruluk Oranı

ABSTRACT

In this study, it is aimed to examine the factors affecting students' academic performance and to predict their mathematics achievement levels. The Students Performance dataset obtained from the Kaggle platform was used in the study. The dataset includes demographic and socio-economic features such as students' gender, parental education level, lunch type, and test preparation status. Mathematics scores were selected as the target variable and categorized into two classes as low and high achievement.During the data preprocessing stage, categorical variables were converted into numerical values, and the dataset was divided into training and testing sets. In the modeling process, K-Nearest Neighbors (KNN), Naive Bayes, Decision Tree, Support Vector Machine (SVM), Logistic Regression, and K-Means algorithms were applied, and their performances were compared. The performance of the models was evaluated using the accuracy metric and confusion matrices. Additionally, the relationships between variables in the dataset were analyzed using correlation analysis.According to the results obtained, the model that achieved the highest accuracy among the applied algorithms was selected as the best-performing model. The study demonstrates that students' mathematics performance can be predicted and that the performance of different machine learning methods can be comparatively evaluated.

Keywords: Student Performance, Data Mining, Machine Learning, Classification, Logistic Regression, Accuracy

1.GİRİŞ

Günümüzde eğitim sistemlerinin temel amaçlarından biri, öğrencilerin akademik başarılarını artırmak ve bu başarıyı etkileyen faktörleri doğru şekilde analiz edebilmektir. Öğrenme, bireyin yaşantıları sonucunda ortaya çıkan nispeten kalıcı davranış değişiklikleri olarak tanımlanmakta ve yalnızca okul ortamıyla sınırlı kalmayıp yaşam boyu devam eden bir süreç olarak kabul edilmektedir. Bu süreçte öğrencilerin akademik başarıları; bireysel, ailesel, çevresel ve sosyo-ekonomik birçok değişkenden etkilenmektedir.

Özellikle modern eğitim sistemlerinde akademik başarı, öğrencilerin gelecekteki eğitim ve kariyer yollarını belirleyen en önemli ölçütlerden biri hâline gelmiştir. Bu durum, öğrencilerin başarı düzeylerini etkileyen faktörlerin bilimsel yöntemlerle incelenmesini gerekli kılmaktadır. Geleneksel istatistiksel yaklaşımlar belirli ölçüde bilgi sağlasa da, günümüzde artan veri miktarı karşısında bu yöntemler yetersiz kalabilmektedir. Bu noktada veri madenciliği ve makine öğrenmesi teknikleri, büyük veri kümeleri içerisindeki gizli örüntülerin keşfedilmesi ve anlamlı tahminlerin yapılması açısından önemli avantajlar sunmaktadır.

Eğitim alanında veri madenciliği yöntemlerinin kullanımı, öğrencilerin başarı ve başarısızlık durumlarının analiz edilmesine, risk altındaki öğrencilerin erken tespit edilmesine ve eğitim politikalarının daha bilinçli şekilde oluşturulmasına katkı sağlamaktadır. Literatürde yapılan çalışmalar, öğrencilerin akademik başarılarının cinsiyet, aile yapısı, ebeveyn eğitim durumu, ekonomik koşullar ve öğrenme alışkanlıkları gibi çok sayıda değişkenden etkilendiğini göstermektedir.

Bu çalışmada, Kaggle platformunda yer alan *Students Performance* veri seti kullanılarak öğrencilerin matematik başarısını etkileyen faktörler incelenmiştir. Veri setinde yer alan demografik ve sosyo-ekonomik değişkenler kullanılarak, öğrencilerin matematik başarıları düşük ve yüksek olmak üzere iki sınıfa ayrılmıştır. Çalışma kapsamında K-En Yakın Komşu (KNN), Naive Bayes, Decision Tree, Support Vector Machine (SVM), Logistic Regression ve KMeans algoritmaları uygulanarak modellerin performansları karşılaştırılmıştır. Elde edilen sonuçlar doğruluk (accuracy) metriği ve hata matrisleri (confusion matrix) aracılığıyla değerlendirilmiştir.

Bu çalışma, farklı makine öğrenmesi algoritmalarının öğrenci başarısı tahminindeki etkinliğini karşılaştırmalı olarak ortaya koymayı ve eğitim alanında veri madenciliği yöntemlerinin kullanımına katkı sağlamayı amaçlamaktadır.

2.MATERYAL VE METODLAR

2.1 Veri Seti (Materyal)

Bu çalışmada kullanılan veri seti, Kaggle platformunda yer alan *Students Performance* veri setidir. Veri seti, öğrencilerin akademik başarılarını etkileyebilecek çeşitli demografik ve sosyo-ekonomik özellikleri içermektedir. Veri setinde; öğrencilerin cinsiyeti, ebeveyn eğitim durumu, öğle yemeği tipi, test hazırlık durumu gibi kategorik değişkenler ile matematik, okuma ve yazma derslerine ait başarı puanları bulunmaktadır.Çalışmada hedef değişken olarak öğrencilerin matematik başarı puanları ele alınmıştır. Matematik puanları, analizlerin sınıflandırma temelli yapılabilmesi amacıyla düşük başarı ve yüksek başarı olmak üzere iki sınıfa ayrılmıştır.

2.2 Veri Ön İşleme

Analiz sürecinden önce veri seti üzerinde ön işleme adımları uygulanmıştır. Veri setinde yer alan kategorik değişkenler, makine öğrenmesi algoritmaları tarafından kullanılabilmesi amacıyla Label Encoding yöntemi ile sayısal değerlere dönüştürülmüştür. Ardından, giriş değişkenleri (X) ve hedef değişken (y) birbirinden ayrılmıştır.Veri seti, modelleme aşamasında tarafsız ve güvenilir sonuçlar elde edebilmek amacıyla %70 eğitim ve %30 test olmak üzere iki alt kümeye ayrılmıştır. Bu işlem, train_test_split fonksiyonu kullanılarak gerçekleştirilmiştir.

2.3 Kullanılan Yöntemler

Çalışmada öğrencilerin matematik başarısını tahmin edebilmek ve farklı yaklaşımların performanslarını karşılaştırabilmek amacıyla çeşitli veri madenciliği ve makine öğrenmesi algoritmaları kullanılmıştır. Bu algoritmalar aşağıda özetlenmiştir:

2.3.1 K-En Yakın Komşu (KNN)

Bu çalışmada KNN algoritması, öğrencilerin demografik ve sosyo-ekonomik özelliklerine göre matematik başarı düzeylerinin (düşük–yüksek) tahmin edilmesi amacıyla kullanılmıştır. Test verilerindeki her öğrenci, eğitim kümesindeki en benzer öğrencilerle karşılaştırılarak sınıflandırılmıştır. KNN algoritmasında öğrenciler arasındaki benzerlik, bir uzaklık metriği aracılığıyla hesaplanmakta olup, bu çalışmada kullanılan mesafe hesaplama yöntemi Denklem 2.1'de sunulmuştur.

$$d(x, x_i) = \sqrt{\sum_{j=1}^{n} (x_j - x_{ij})^2}$$

Denklem 2.1: K-En Yakın Komşu Algoritması

2.3.2 Naive Bayes

Naive Bayes algoritması, öğrencilerin özellikleri ile matematik başarı düzeyleri arasındaki olasılıksal ilişkileri modellemek amacıyla uygulanmıştır. Değişkenler arası bağımsızlık varsayımı altında başarı tahmini yapılmış ve diğer modellerle performans karşılaştırması gerçekleştirilmiştir. Bu kapsamda, her bir öğrencinin başarı sınıfına ait olma olasılığı Bayes teoremi kullanılarak hesaplanmış olup, kullanılan olasılıksal sınıflandırma kuralı Denklem 2.2'de verilmiştir.

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Denklem 2.2:Bayes Teoremi

2.3.3 Decision Tree (Karar Ağacı)

Karar Ağacı yöntemi, öğrencilerin matematik başarısını etkileyen karar kurallarını belirlemek ve bu kuralları görsel olarak incelemek amacıyla kullanılmıştır. Elde edilen ağaç yapısı, değişkenlerin sınıflandırma sürecindeki rolünü yorumlamaya olanak sağlamıştır. Bu süreçte sınıflandırma, düğümlerde gerçekleştirilen bölme işlemleriyle bilgi kazancının (Information Gain) maksimize edilmesi esasına dayanmakta olup, kullanılan karar ölçütü Denklem 2.3'te sunulmuştur.

$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2 p_i$$

$$IG(S, A) = Entropy(S) - \sum_{v \in A} \frac{|S_v|}{|S|} Entropy(S_v)$$

Denklem 2.3: Decision Tree-Entropi/Information Gain

2.3.4 Support Vector Machine (SVM)

SVM algoritması, düşük ve yüksek başarıya sahip öğrencileri en iyi şekilde ayıran karar sınırını belirlemek amacıyla kullanılmıştır. Modelin başarımı doğruluk metriği ile diğer yöntemlerle karşılaştırılmıştır. Bu kapsamda, sınıflar arasındaki ayırımı maksimize eden optimal hiper düzlem, marjin genişliğini en üst düzeye çıkaracak şekilde belirlenmiş olup, sınıflandırma kuralı Denklem 2.4'te verilmiştir.

$$w \cdot x + b = 0$$

$$\min \frac{1}{2}||w||^2$$

$$y_i(w \cdot x_i + b) \geq 1$$

Denklem 2.4:SVM Algoritması

2.3.5 Logistic Regression (Lojistik Regresyon)

Lojistik Regresyon yöntemi, öğrencilerin matematik başarısını ikili sınıflandırma problemi olarak ele alarak başarı olasılıklarını hesaplamak için kullanılmıştır. Çalışmada en yüksek doğruluk oranını sağlayan modellerden biri olarak değerlendirilmiştir. Bu yöntemde, giriş değişkenleri ile başarı olasılığı arasındaki ilişki sigmoid (lojistik) fonksiyonu kullanılarak modellenmiş olup, sınıflandırma kuralı Denklem 2.5'te sunulmuştur.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$
$$z = w^T x + b$$

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

$$\hat{y} = \begin{cases} 1, & P \geq 0.5 \\ 0, & P < 0.5 \end{cases}$$

Denklem 2.5:Logistik Regression Algoritması

2.3.6 KMeans

KMeans algoritması, denetimsiz öğrenme yaklaşımıyla öğrencileri iki kümeye ayırmak amacıyla kullanılmıştır. Oluşturulan kümeler, gerçek başarı sınıflarıyla karşılaştırılarak kümeleme sonuçlarının sınıflandırma performansına yakınlığı analiz edilmiştir. Bu yöntemde, kümeleme işlemi küme içi hata kareleri toplamını minimize edecek şekilde gerçekleştirilmiş olup, kullanılan fonksiyon Denklem 2.6'da verilmiştir.

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Denklem 2.6:KMeans Algoritması

2.4 Yöntemlerin Karşılaştırılması ve En İyi Modelin Belirlenmesi

Bu çalışmada, öğrencilerin matematik başarı düzeylerini tahmin edebilmek amacıyla farklı makine öğrenmesi ve veri madenciliği yöntemleri uygulanmış ve bu yöntemlerin performansları karşılaştırmalı olarak değerlendirilmiştir. Tüm modeller, aynı veri seti ve aynı eğitim–test bölünmesi kullanılarak eğitilmiş ve doğruluk (accuracy) metriği ile hata matrisleri (confusion matrix) üzerinden değerlendirilmiştir. Bu sayede yöntemler arasında adil ve tutarlı bir karşılaştırma yapılması sağlanmıştır.

K-En Yakın Komşu (KNN) algoritması, benzer öğrenciler arasındaki mesafe ilişkilerini temel alarak sınıflandırma yapmış ve yerel benzerliklere dayalı sonuçlar üretmiştir. Naive Bayes yöntemi, olasılık temelli yaklaşımı sayesinde hızlı sonuçlar sunmuş; ancak değişkenler arasındaki bağımsızlık varsayımı nedeniyle bazı durumlarda sınırlı performans göstermiştir. Decision Tree algoritması, karar kuralları oluşturarak öğrencilerin matematik başarısını etkileyen değişkenlerin yorumlanmasına olanak sağlamış, ancak karmaşık yapılarda aşırı öğrenme eğilimi gösterebilmiştir. Support Vector Machine (SVM) yöntemi ise sınıflar arasındaki ayırımı güçlü bir karar sınırı ile gerçekleştirmiştir.

Logistic Regression algoritması, ikili sınıflandırma problemine uygun yapısı sayesinde öğrencilerin matematik başarısını tahmin etmede dengeli ve kararlı sonuçlar üretmiştir. Modelin doğrusal yapısı, veri setindeki değişkenler ile hedef değişken arasındaki ilişkileri etkili biçimde yakalayabilmiştir. Ayrıca modelin aşırı karmaşık olmaması, test verisi üzerinde daha yüksek genellenebilirlik sağlamıştır.

Bu nedenlerle Logistic Regression, diğer yöntemlere kıyasla en yüksek doğruluk oranını elde ederek çalışmada en iyi performans gösteren model olarak belirlenmiştir.

Denetimsiz öğrenme yöntemi olan KMeans algoritması ise öğrencileri benzer özelliklere göre iki kümeye ayırmış ve elde edilen kümeler, gerçek başarı sınıflarıyla karşılaştırılarak kümeleme sonuçlarının sınıflandırma performansına ne ölçüde yaklaştığı analiz edilmiştir. KMeans sonuçları, denetimli öğrenme yöntemleriyle elde edilen sonuçlarla destekleyici bir bakış açısı sunmuştur.

Sonuç olarak, farklı makine öğrenmesi algoritmalarının öğrenci başarısı tahminindeki etkinliği karşılaştırmalı olarak incelenmiş ve Logistic Regression modelinin bu veri seti için en uygun yöntem olduğu belirlenmiştir. Bu bulgular, eğitim alanında veri madenciliği yöntemlerinin başarı tahmini ve karar destek süreçlerinde etkili bir şekilde kullanılabileceğini göstermektedir.

3.SONUÇLAR VE BULGULAR

Bu çalışmada, öğrencilerin matematik başarı düzeylerini tahmin edebilmek amacıyla farklı makine öğrenmesi ve veri madenciliği yöntemleri uygulanmış ve elde edilen sonuçlar karşılaştırmalı olarak analiz edilmiştir. Students Performance veri seti üzerinde gerçekleştirilen deneylerde, tüm modeller aynı eğitim ve test veri kümeleri kullanılarak eğitilmiş ve değerlendirilmiştir. Böylece yöntemler arasında adil ve tutarlı bir performans karşılaştırması yapılması sağlanmıştır.

Uygulanan modellerin performansları doğruluk (accuracy) metriği Şekil 3.3 ve hata matrisleri (confusion matrix) Şekil 3.1 aracılığıyla değerlendirilmiştir. Elde edilen sonuçlar, kullanılan algoritmalar arasında belirgin performans farklılıkları olduğunu göstermiştir. K-En Yakın Komşu (KNN), Naive Bayes, Decision Tree ve Support Vector Machine (SVM) modelleri belirli bir doğruluk seviyesine ulaşmış olsa da, bu modellerin bazıları veri yapısına bağlı olarak sınırlı genellenebilirlik göstermiştir. Özellikle Decision Tree algoritması yorumlanabilir bir yapı sunmasına rağmen Şekil 3.2'den de gözlemleyebildiğimiz üzere karmaşık karar yapılarında aşırı öğrenmeye yatkınlık sergileyebilmiştir.

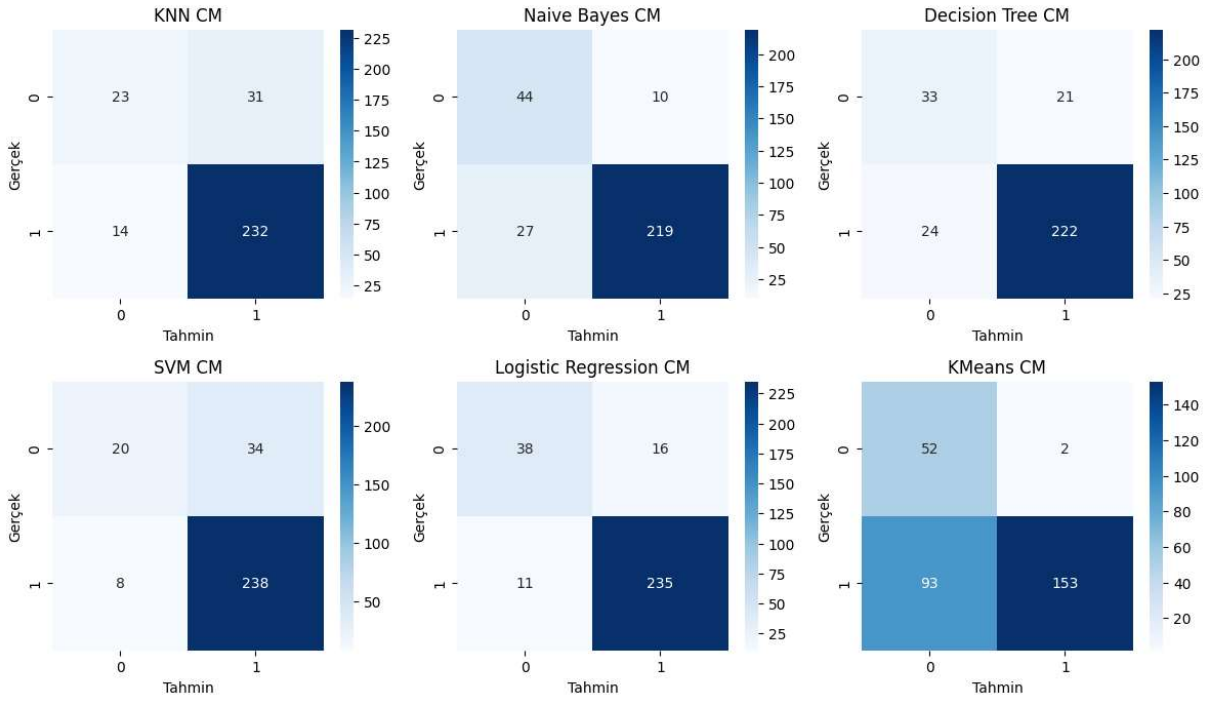
Logistic Regression algoritması ise diğer yöntemlere kıyasla Şekil 3.4'te hesaplanarak gösterildiği gibi en yüksek doğruluk oranını elde etmiş ve test verisi üzerinde daha dengeli sonuçlar üretmiştir. Modelin ikili sınıflandırma problemine uygun yapısı ve aşırı karmaşık olmaması, genellenebilirliğinin yüksek olmasını sağlamıştır. Bu nedenle Logistic Regression, çalışmada en iyi performans gösteren model olarak belirlenmiştir.

En iyi model için oluşturulan hata matrisi incelendiğinde, düşük ve yüksek başarı sınıflarının büyük ölçüde doğru tahmin edildiği görülmüştür.

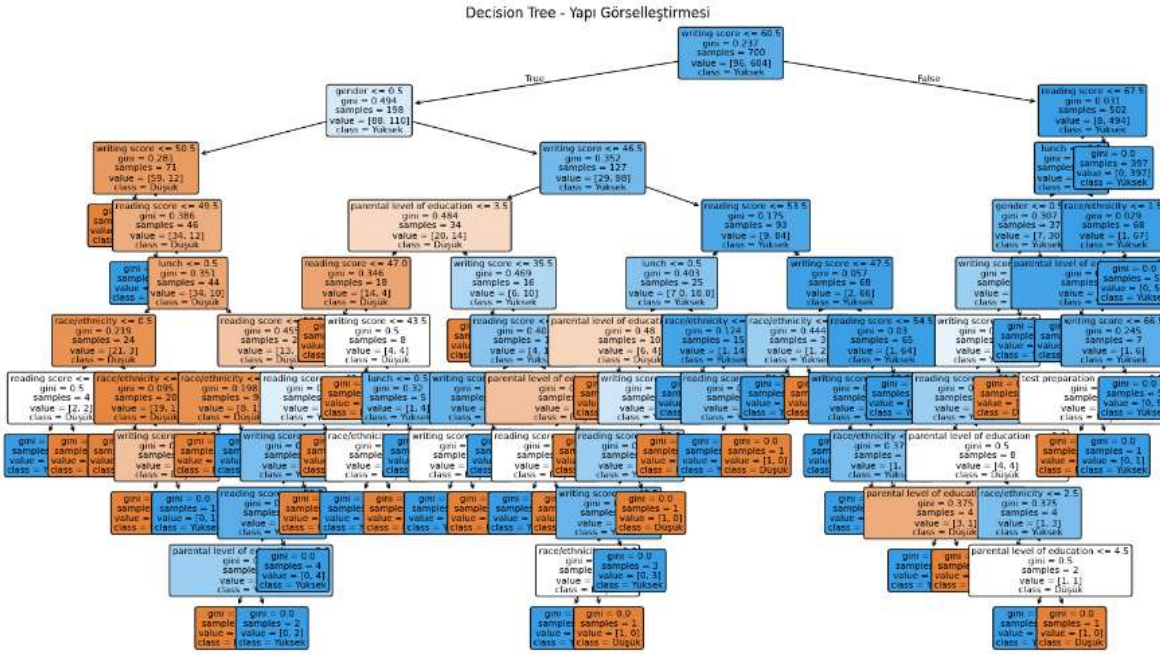
Denetimsiz öğrenme yöntemi olan KMeans algoritması ile öğrenciler benzer özelliklerine göre iki kümeye ayrılmış ve elde edilen kümeler gerçek başarı sınıflarıyla karşılaştırılmıştır. Kümeleme sonuçlarının sınıflandırma modellerine kıyasla daha düşük performans gösterdiği, ancak veri yapısının genel eğilimleri hakkında destekleyici bilgiler sunduğu belirlenmiştir. PCA yöntemi kullanılarak iki boyuta indirgenen veri üzerinde yapılan görselleştirmeler, kümeler arasındaki ayrımın kısmen gözlemlenebilir olduğunu Şekil 3.6'da da ortaya koymuştur.

Ayrıca korelasyon analizi sonuçları Şekil 3.5'te görülmek üzere matematik başarısı ile bazı demografik ve sosyo-ekonomik değişkenler arasında anlamlı ilişkiler bulunduğunu göstermiştir. Bu durum, öğrencilerin akademik başarısının yalnızca bireysel değil, çevresel ve ailesel faktörlerden de etkilendiğini desteklemektedir.

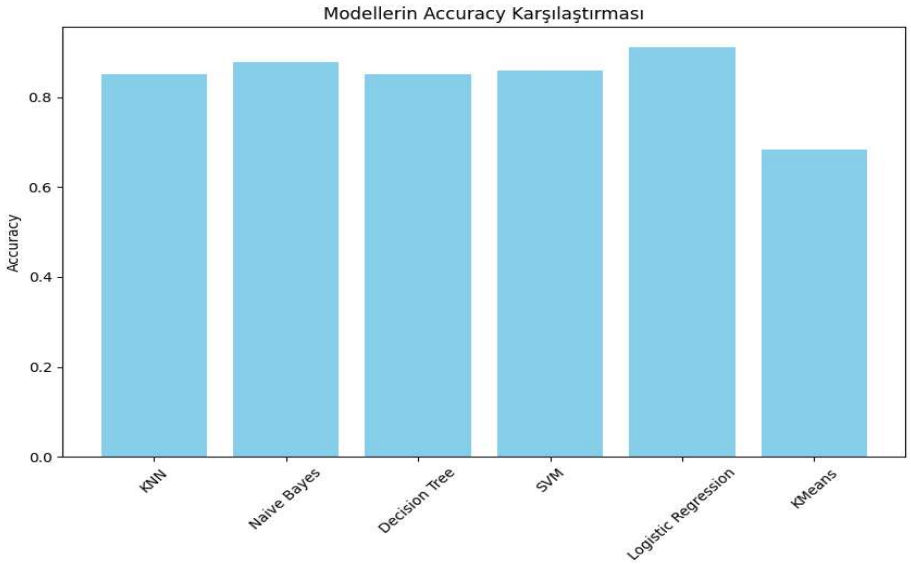
Sonuç olarak, bu çalışma kapsamında uygulanan makine öğrenmesi yöntemleri ile öğrencilerin matematik başarı düzeylerinin tahmin edilebileceği gösterilmiştir. Özellikle Logistic Regression modelinin bu veri seti için en uygun yöntem olduğu sonucuna ulaşılmıştır. Elde edilen bulgular, eğitim alanında veri madenciliği ve makine öğrenmesi yöntemlerinin karar destek süreçlerinde etkili bir şekilde kullanılabileceğini ortaya koymaktadır.



Şekil 3.1:Tüm Modellerin Confusion Matrixleri



Şekil 3.2:Desicion Tree Görselleştirmesi



Şekil 3.3: Tüm Modellerin Accuracy Karşılaştırması

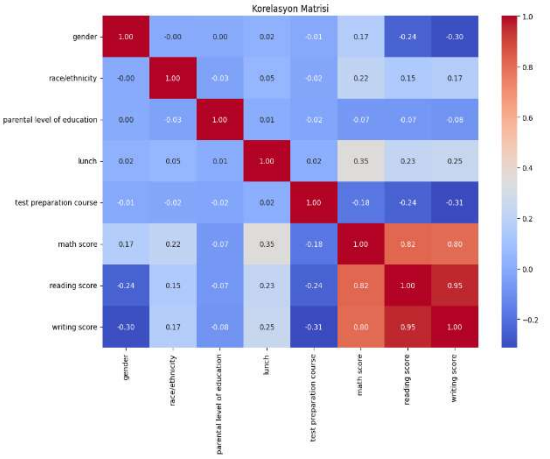
```
--- GRAFİKLİ TAM MODEL KARŞILAŞTIRMA SİSTEMİ ---

KNN → Accuracy = 0.8500
Naive Bayes → Accuracy = 0.8767
Decision Tree → Accuracy = 0.8533
SVM → Accuracy = 0.8600
Logistic Regression → Accuracy = 0.9100
KMeans → Accuracy = 0.6833

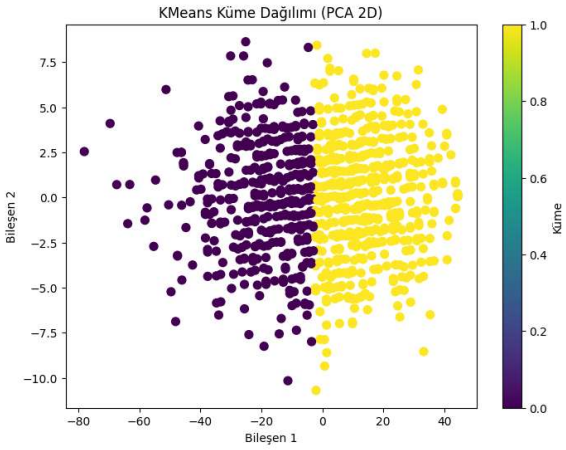
--- MODELLERİN SIRALAMASI (En İyiden → En Kötüye) ---
Logistic Regression > Naive Bayes > SVM > Decision Tree > KNN > KMeans

------------------------------------
🚀 EN İYİ MODEL: Logistic Regression
------------------------------------
```

Şekil 3.4:Tüm Modellerin Sayısal Sonuçlarının Karşılaştırılıp En İyi Modelin Seçilmesi



Şekil 3.5:Korelasyon Matrisi



Şekil 3.6:KMeans Küme Dağılımı

KAYNAKÇA

1.https://dergipark.org.tr/en/download/article-file/3876452

2.https://www.kaggle.com/datasets/sadiajavedd/students-academic-performance-dataset

3.https://colab.research.google.com/drive/1wbzdtON89T_Ue6nVPAqn1ysoT2o-d38b

4.https://github.com/zeynep-png/zeynep-png/commits?author=zeynep-png

5.https://dergipark.org.tr/tr/pub/tutad/article/885812

6.https://dergipark.org.tr/en/pub/ticaretfbd/issue/84890/1442084

7.https://dergipark.org.tr/en/download/article-file/456213

8.Koç, N., Liselerde öğrencilerin akademik başarılarının değerlendirilmesi uygulamalarının etkinliğine ilişkin bir araştırma. Eğitim ve Bilim. 28-36. (doktora tezi 1977; ankara: eğitim fakültesi yay. 1981

9.https://www.indexacademicdocs.org/pdf/56/62829/865162

10. https://www.kaggle.com/datasets/kundanbedmutha/student-performance-dataset

EK-1 Python ile Veri Ön İşleme ve Modelleme Kodu (Açıklamalı)

1. Kullanılan Kütüphaneler

Bu bölümde veri işleme, makine öğrenmesi modelleri ve görselleştirme için gerekli kütüphaneler tanımlanmıştır.

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from sklearn.neighbors import KNeighborsClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.svm import SVC

from sklearn.cluster import KMeans

from sklearn.linear_model import LogisticRegression

from sklearn.decomposition import PCA

from scipy.optimize import linear_sum_assignment

import matplotlib.pyplot as plt

import seaborn as sns
```

2. KMeans İçin Etiket Düzeltme Fonksiyonu

KMeans algoritması kümelere rastgele etiket atadığı için, gerçek sınıflarla uyum sağlamak amacıyla Hungarian algoritması kullanılarak etiketler düzeltilmiştir.

```
def fix_kmeans_labels(y_true, y_pred):
    matrix = np.zeros((2, 2))
    for i in range(2):
        for j in range(2):
            matrix[i, j] = np.sum((y_true == i) & (y_pred == j))
```

```
    row_ind, col_ind = linear_sum_assignment(matrix.max() - matrix)
    mapping = {col_ind[i]: row_ind[i] for i in range(len(row_ind))}
    return np.array([mapping[label] for label in y_pred])
```

3. Veri Setinin Yüklenmesi

Çalışmada Kaggle platformundan alınan *StudentsPerformance.csv* veri seti kullanılmıştır.

```
df = pd.read_csv("StudentsPerformance.csv")
```

4. Veri Ön İşleme

Kategorik değişkenler, makine öğrenmesi algoritmalarında kullanılabilmesi için sayısal değerlere dönüştürülmüştür.

```
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
```

5. Bağımlı ve Bağımsız Değişkenlerin Belirlenmesi

Hedef değişken olarak matematik notu seçilmiş ve düşük–yüksek başarı olacak şekilde iki sınıfa ayrılmıştır.

```
X = df.drop("math score", axis=1)
y = df["math score"]
y = pd.cut(y, bins=2, labels=[0, 1]).astype(int)
```

6. Eğitim ve Test Verilerinin Ayrılması

Veri seti %70 eğitim ve %30 test olacak şekilde ayrılmıştır.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

7. Kullanılan Makine Öğrenmesi Modelleri

Çalışmada farklı denetimli ve denetimsiz öğrenme algoritmaları kullanılmıştır.

```
models = {
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(),
    "SVM": SVC(),
```

```
    "Logistic Regression": LogisticRegression(max_iter=300),
    "KMeans": KMeans(n_clusters=2, random_state=42)
}
```

8. Modellerin Eğitilmesi ve Test Edilmesi

Tüm modeller aynı eğitim ve test verileriyle eğitilmiş ve doğruluk (accuracy) metriği ile değerlendirilmiştir.

```
results = {}
all_conf_matrices = {}
for name, model in models.items():
    if name == "KMeans":
        model.fit(X_train)
        preds = model.predict(X_test)
        preds = fix_kmeans_labels(y_test.values, preds)
    else:
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    results[name] = acc
    all_conf_matrices[name] = confusion_matrix(y_test, preds)
```

9. Modellerin Performansa Göre Sıralanması

Modeller doğruluk değerlerine göre büyükten küçüğe sıralanmıştır.

```
sorted_model_names = [
    name for name, acc in sorted(results.items(), key=lambda x: x[1], reverse=True)
]
```

10. En İyi Modelin Seçilmesi

En yüksek doğruluk değerine sahip model “en iyi model” olarak belirlenmiştir.

```
best_model_name = sorted_model_names[0]
best_model = models[best_model_name]
```

11. Sonuçların Görselleştirilmesi

Modellerin doğruluk karşılaştırması,Korelasyon matrisi,Confusion matrix’ler,Decision Tree yapısı, KMeans küme dağılımı (Bu grafikler çalışmanın deneysel bulgularını desteklemek amacıyla kullanılmıştır.)

plt.bar(results.keys(), results.values())

plt.show()

12. Sınıflandırma Raporu

En iyi model için precision, recall ve F1-score değerleri raporlanmıştır.

print(classification_report(y_test, final_preds))

EK-2. Python ile Veri Ön İşleme ve Modelleme Kodu

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from scipy.optimize import linear_sum_assignment
import matplotlib.pyplot as plt
import seaborn as sns
# ------------------------------
#   KMEANS LABEL FIX
# ------------------------------
def fix_kmeans_labels(y_true, y_pred):
    matrix = np.zeros((2, 2))
    for i in range(2):
        for j in range(2):
            matrix[i, j] = np.sum((y_true == i) & (y_pred == j))
    row_ind, col_ind = linear_sum_assignment(matrix.max() - matrix)
    mapping = {col_ind[i]: row_ind[i] for i in range(len(row_ind))}
    return np.array([mapping[label] for label in y_pred])
# ------------------------------
#   MAIN
```

```
# ------------------------------
def main():
    print("\n--- GRAFİKLİ TAM MODEL KARŞILAŞTIRMA SİSTEMİ ---\n")
    # --- VERİ ---
    df = pd.read_csv("StudentsPerformance.csv")
    # string kolonları sayıya çevir
    le = LabelEncoder()
    for col in df.columns:
        if df[col].dtype == 'object':
            df[col] = le.fit_transform(df[col])
    # X - y ayır
    X = df.drop("math score", axis=1)
    y = df["math score"]
    # math score → kategori (0 = düşük, 1 = yüksek)
    y = pd.cut(y, bins=2, labels=[0, 1]).astype(int)

    # Veri bölme
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
    # ------------------------------
    # MODELLER
    # ------------------------------
    models = {
        "KNN": KNeighborsClassifier(),
        "Naive Bayes": GaussianNB(),
        "Decision Tree": DecisionTreeClassifier(),
        "SVM": SVC(),
        "Logistic Regression": LogisticRegression(max_iter=300),
        "KMeans": KMeans(n_clusters=2, random_state=42)
    }
    results = {}
    all_conf_matrices = {}
    # ------------------------------
    # TÜM MODELLERİ EĞİT-TEST
```

```
# ------------------------------
for name, model in models.items():
    if name == "KMeans":
        model.fit(X_train)
        preds = model.predict(X_test)
        preds = fix_kmeans_labels(y_test.values, preds)
    else:
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    results[name] = acc
    all_conf_matrices[name] = confusion_matrix(y_test, preds)
    print(f"{name} → Accuracy = {acc:.4f}")
# ------------------------------
#   MODEL SIRALAMA (EKLEDİĞİM KISIM)
# ------------------------------
sorted_model_names = [name for name, acc in sorted(results.items(), key=lambda x: x[1], reverse=True)]
ranking_line = " > ".join(sorted_model_names)
print("\n--- MODELLERİN SIRALAMASI (En İyiden → En Kötüye) ---")
print(ranking_line)
# ------------------------------
#   EN İYİ MODELİ SEÇ
# ------------------------------
best_model_name = sorted_model_names[0]
print("\n-------------------------------------")
print(f"🚀 EN İYİ MODEL: {best_model_name}")
print("-------------------------------------")
best_model = models[best_model_name]
if best_model_name == "KMeans":
    best_model.fit(X_train)
    final_preds = best_model.predict(X_test)
    final_preds = fix_kmeans_labels(y_test.values, final_preds)
else:
```

```
    best_model.fit(X_train, y_train)
    final_preds = best_model.predict(X_test)
# ------------------------------
# GRAFİK 1 → ACCURACY BAR CHART
# ------------------------------
plt.figure(figsize=(10, 6))
plt.bar(results.keys(), results.values(), color="skyblue")
plt.title("Modellerin Accuracy Karşılaştırması")
plt.ylabel("Accuracy")
plt.xticks(rotation=45)
plt.show()
# ------------------------------
# GRAFİK 2 → KORELASYON HEATMAP
# ------------------------------
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Korelasyon Matrisi")
plt.show()
# ------------------------------
# GRAFİK 3 → TÜM MODELLERİN CONFUSION MATRIX'İ
# ------------------------------
plt.figure(figsize=(12, 10))
index = 1
for name, cm in all_conf_matrices.items():
    plt.subplot(3, 3, index)
    sns.heatmap(cm, annot=True, cmap="Blues", fmt="d")
    plt.title(f"{name} CM")
    plt.xlabel("Tahmin")
    plt.ylabel("Gerçek")
    index += 1
plt.tight_layout()
plt.show()
# ------------------------------
```

```
# GRAFİK 4 → EN İYİ MODELİN BÜYÜK CM
# ------------------------------
cm_best = confusion_matrix(y_test, final_preds)
plt.figure(figsize=(6, 5))
sns.heatmap(cm_best, annot=True, cmap="Blues", fmt="d")
plt.title(f"En İyi Model: {best_model_name} - Confusion Matrix")
plt.xlabel("Tahmin")
plt.ylabel("Gerçek")
plt.show()
# ------------------------------
# EK GRAFİK → DECISION TREE GÖRSELİ
# ------------------------------
dt_model = models["Decision Tree"]
plt.figure(figsize=(18, 10))
plot_tree(dt_model, feature_names=X.columns, class_names=["Düşük", "Yüksek"],
          filled=True, rounded=True, fontsize=8)
plt.title("Decision Tree - Yapı Görselleştirmesi")
plt.show()
# ------------------------------
# GRAFİK 5 → KMEANS KÜME DAĞILIMI (PCA 2D)
# ------------------------------
if "KMeans" in models:
    pca = PCA(n_components=2)
    X_2D = pca.fit_transform(X)
    kmeans = KMeans(n_clusters=2, random_state=42)
    clusters = kmeans.fit_predict(X)
    clusters_fixed = fix_kmeans_labels(y.values, clusters)
    plt.figure(figsize=(8, 6))
    plt.scatter(X_2D[:, 0], X_2D[:, 1], c=clusters_fixed, cmap="viridis", s=50)
    plt.title("KMeans Küme Dağılımı (PCA 2D)")
    plt.xlabel("Bileşen 1")
    plt.ylabel("Bileşen 2")
    plt.colorbar(label="Küme")
```

```
        plt.show()
    # ------------------------------
    # CLASSIFICATION REPORT
    # ------------------------------
    print("\nCLASSIFICATION REPORT:")
    print(classification_report(y_test, final_preds))
# RUN
main()
```